

Devang Jagdish Jalag

[LinkedIn](#) | [GitHub](#) | devangjalag@gmail.com | +1 647-707-7839 | Halifax, NS

Professional Summary

Master of Applied Computer Science student with hands-on experience designing AWS-native serverless backends via Terraform IaC, building Python and Java distributed systems, and shipping full-stack React/Flask applications integrating LLM APIs. Comfortable across the SDLC: defense-in-depth IAM design, automated testing, peer code reviews, and incident-aware deployment. Actively seeking a Fall 2026 Software Engineering Co-op.

Education

Dalhousie University

Sep 2025 - Dec 2026 (Expected)

Master of Applied Computer Science (MACS) | CGPA: 4.1/4.3

L.D. College of Engineering (GTU)

Oct 2021 - May 2025

Bachelor of Engineering, Computer Engineering | CGPA: 8.87/10 | **Advanced Coursework:** 50-Hr Image Processing & CV (IIT Gandhinagar PMRF)

Skills

Languages: Python, Java, JavaScript

Cloud & Infrastructure: AWS Lambda, AWS S3, AWS API Gateway (HTTP v2), AWS CloudFront, AWS Cognito, AWS IAM, AWS ECS Fargate, AWS EventBridge, AWS CloudWatch, Origin Access Control (OAC), Terraform, Firebase Storage

Databases: AWS DynamoDB, PostgreSQL, Firebase Firestore

Frameworks & Libraries: Flask, SQLAlchemy, boto3, Leaflet.js, Google Gemini 2.5 Flash API, google-genai SDK, bleach, BeautifulSoup, PyJWT, Jackson JSON, jline, expo-location

Work Experience

Palm Infotech | *Software Developer Intern*
Surat, India

Jan 2025 - Apr 2025

- Engineered a full-stack mobile lottery application, improving UI rendering speed and session retention, by developing a cross-platform Flutter frontend integrated with GetStorage and advanced image caching techniques.
- Accelerated lottery matching logic and result processing workflows, reducing data retrieval latency by ~25%, by architecting robust NestJS REST APIs and implementing complex MongoDB aggregation pipelines alongside MySQL schemas.
- Strengthened system architecture and API request validation, ensuring stable data handoffs across backend services, by implementing NestJS Data Transfer Objects (DTOs), validation pipes, and secure authentication flows.

Academic Projects

CloudNotes | *Software Engineer (Academic)* | Dalhousie University

Jan 2026 - Apr 2026

- Engineered a direct-to-S3 image upload pipeline using SigV4 pre-signed PUT URLs (300-second TTL) generated by Lambda, bypassing API Gateway's 10 MB and Lambda's 6 MB payload limits; atomically appended object keys to DynamoDB note records using a `list_append(if_not_exists())` update expression — enabling binary uploads without increasing Lambda memory or timeout, on Python + boto3 + DynamoDB + S3.`
- Implemented defense-in-depth access control across 3 layers via Terraform IaC: CloudFront Origin Access Control (OAC, SigV4-signed) as the sole authorized S3 principal with all public ACLs blocked; API Gateway HTTP v2 JWT authorizer validating Cognito tokens natively (eliminating a custom Lambda authorizer); and a least-privilege IAM execution role scoped to 7 DynamoDB actions across 3 named tables — using Cognito, API Gateway, Lambda, and S3.
- Built an immutable note version-history system that snapshots full note state to a dedicated DynamoDB table on every PUT, sorted descending by ISO timestamp range key (ScanIndexForward=False), capped at `MAX_VERSIONS=10` via Lambda env var; paired with a UUID-hex share-token system using DynamoDB TTL (7-day `expires_at` plus an application-layer expiry guard to handle DynamoDB TTL sweep lag — using Python Lambda + DynamoDB + API Gateway HTTP v2.`

DalParkAid | *Software Engineer (Academic)* | Dalhousie University

Jan 2026 - Apr 2026

- Built a three-signal parking prediction engine fusing time/weather/academic-calendar heuristics, crowdsourced reports weighted by exponential freshness decay (45-min decay constant, 90-min cutoff window), and a 42-day historical pattern model — generating a confidence-ranked, hysteresis-smoothed 24-hour forecast across 28 lots.

- Enforced submission data quality using haversine-formula proximity validation (30m geofence via expo-location) and hardened cloud uploads via Firebase Storage security rules scoping writes to authenticated, UID-partitioned, image-only paths capped at 10 MB.
- Maintained cross-client consistency across four concurrent Firestore onSnapshot subscriptions using atomic read-modify-write transactions for community vote counts — feeding a dynamically-recomputed trust multiplier (0.35–1.90 range) back into the prediction pipeline.

Interactive World Map | *Software Engineer (Academic)* | Dalhousie University Mar 2026

- Enforced zero-trust S3 access via CloudFront Origin Access Control (OAC) with SigV4 signing — all public bucket ACLs blocked in Terraform (main.tf:25-32), CloudFront as sole authorized principal, HTTPS redirect enforced at CDN layer (main.tf:104).
- Real-time GeoJSON layer re-styling on every keystroke — input events trigger `geojsonLayer.setStyle()` across all rendered features, with a 3-state style resolver (selected / search-highlighted / default) computed in O(1) via JS Map lookups, using Leaflet.js + vanilla JavaScript.
- Deterministic 30-color palette assignment (index.html:83-89) with `etag = filemd5()` cache-busting in Terraform (main.tf:64) — ensures zero stale-asset serving on every deployment, using AWS CloudFront + S3 + Terraform IaC.

SecureShare | *Software Engineer (Academic)* | Dalhousie University Jan 2026 - Apr 2026

- Chained API Gateway's native JWT authorizer against an Amazon Cognito User Pool with application-layer ``user_id`` claim extraction in Lambda — enforcing per-user data isolation in DynamoDB so no authenticated principal can enumerate another user's file metadata, using Cognito, API Gateway HTTP v2, and Python Lambda.
- Implemented defense-in-depth S3 access control by enabling all four Block Public Access settings, AES256 server-side encryption, and 3600-second presigned URL expiry — eliminating all publicly addressable file endpoints while supporting authenticated uploads and unauthenticated share-link downloads via server-side TTL validation.
- Designed an ephemeral ECS Fargate janitor triggered by Amazon EventBridge ``rate(1 hour)``, scanning DynamoDB for expired ``expires_at`` entries and issuing targeted ``s3:DeleteObject`` calls — preventing orphaned object accumulation with a cost model that charges only for active task execution seconds, backed by 7-day CloudWatch log retention.

DalOverflow | *Software Engineer (Academic)* | Dalhousie University Sep 2025 - Dec 2025

- Integrated Google Gemini 2.5 Flash API for AI-powered Q&A answer generation and multi-answer summarization; implemented truncation-aware two-pass retry — detecting MAX_TOKENS via FinishReason and re-invoking with a condensed prompt at a 10,000-token ceiling — using google-genai SDK on a Flask/PostgreSQL backend.
- Built layered stored-XSS defense using Python bleach (13-tag allowlist, 3 attribute configs) with a regex pre-filter stripping ``<script>`` injection; enforced at model creation and PUT edits via SQLAlchemy hooks, with BeautifulSoup plaintext extraction validating a 20-character minimum body constraint.
- Designed a reusable JWT (PyJWT / HS256) Flask decorator supporting dual API/session auth routing; applied optimistic concurrency control on question edits using client-supplied ``last_known_update`` timestamps — returning HTTP 409 Conflict on stale writes to prevent silent data loss under concurrent edits.

Distributed DBMS Prototype | *Software Engineer (Academic)* | Dalhousie University Sep 2025 - Dec 2025

- Implemented a coordinator-worker distributed DBMS in Java supporting range-based row partitioning and vertical column partitioning across 2 worker nodes, with per-worker LinkedBlockingQueue + CompletableFuture dispatch and automatic schema-signature detection that remaps partition tables on schema drift — using Java sockets, Jackson JSON, and jline.
- Engineered distributed atomicity with 2PC-style coordination: coordinator issues BEGIN/COMMIT/ROLLBACK across all participant workers per DML operation; any failure triggers doRollbackAll() across active workers, preserving cross-node consistency; selective replication (e.g. transaction and permit tables replicated to both workers simultaneously) — using ReentrantLock-guarded execution and CompletableFuture.get().
- Built a custom multi-client TCP database server with wound-wait deadlock detection, per-session Thread dispatch, ReentrantLock-serialized message handling, implicit transaction wrapping for single DML statements, PBKDF2-HMAC-SHA256 password hashing (100 iterations), and a B+ tree index layer for range queries — supporting CRUD, DDL, and ACID transactions across concurrent sessions.

Certificates

AWS Solutions Architect – Associate (In Progress), AWS Cloud Foundations, GCP, IBM SkillsBuild Data & AI, Image Processing & CV (IIT Gandhinagar PMRF)